

FindService fully. The only task remaining is to expose it as a REST service.

The REST API

Our *FindService* offers only one API and that is to find a user by name. The obvious URI is as follows:

```
GET /user/{name}
```

This API is expected to find the user with the supplied name and returns the details of phone number, etc, of the user in JSON format. In case no such user is found, the API is expected to return a 404 status.

We can use the Flask framework to build the REST API. This framework was originally built for developing Web applications using Python. It is further extended to support the REST views besides the HTML views. We pick this framework for its simplicity.

Start by creating a Flask application:

```
from flask import Flask
app = Flask(__name__)
```

Then define the routes for the Flask application as simple functions:

```
@app.route('/user/<name>')
def get(name): pass
```

Observe that the `@app.route` is mapped to the API `/user/<name>` on one side and to the function `get()` on the other side.

As you may already have figured out, this `get()` function will be invoked every time the user accesses the API with a URI like `http://server:port/user/Krishna`. Flask is intelligent enough to extract 'Krishna' as the name from the URI and pass it to the `get()` function.

The `get()` function is straightforward. It asks the controller to find the user and returns the same after marshalling it to the JSON format along with the usual HTTP headers. In case the controller returns *None*, the `get()` function returns the response with an appropriate HTTP status.

```
from flask import jsonify, abort

controller = UserController()
record = controller.find(name)
if record is None:
    abort(404)
else:
    resp = jsonify(record.toJSON())
    resp.status_code = 200
```

```
return resp
```

And, finally, the Flask app needs to be served. We can use the *waitress* server for this purpose.

```
from waitress import serve
serve(app, host="0.0.0.0", port=8080)
```

In the above snippet, the app is served on port 8080 on the local host.

The final code looks like this:

```
from flask import Flask, jsonify, abort
from waitress import serve

app = Flask(__name__)

@app.route('/user/<name>')
def get(name):
    controller = UserController()
    record = controller.find(name)
    if record is None:
        abort(404)
    else:
        resp = jsonify(record.toJSON())
        resp.status_code = 200
        return resp

serve(app, host="0.0.0.0", port=8080)
```

Deployment

The *FindService* is now ready with code. It has its domain model, data layer, and application layer besides the REST API. The next step is to build the service, containerise it, and then deploy it on Kubernetes. The process is in no way different from any other service, though there are some Python-specific steps.

Before proceeding further, let us have a look at the folder/file structure:

```
+ ums-find-service
+ ums
  - domain.py
  - data.py
  - app.py
  - Dockerfile
  - requirements.txt
  - kube-find-deployment.yml
```

As you can observe, the whole work is under the *ums-find-service* folder. It contains the code in the *ums* folder and configurations in *Dockerfile*, *requirements.txt* and *kube-find-deployment.yml* files.